

Πρώτα βήματα στη MongoDB

- 1 Εγκατάσταση της MongoDB και του mongosh, τώρα, στον υπολογιστή σας.
- 2 Οι έννοιες: βάση, συλλογή, έγγραφο· BSON και τύποι δεδομένων.
- 3 Ο σχεδιασμός: ενσωμάτωση ή αναφορά.
- 4 Οι πρώτες εντολές στο mongosh, και τι καλύπτουν από την Εργασία 2.

Κάθε εντολή εμφανίζεται με την έξοδό της· στις διαφάνειες «Δοκιμάστε» την εκτελείτε κι εσείς.

Εγκατάσταση: τι χρειάζεστε

MongoDB Community Server

mongodb.com/try/download/community: Windows, πακέτο msi. Στον οδηγό: Complete και «Install MongoDB as a Service». Το Compass είναι προαιρετικό.

mongosh

mongodb.com/try/download/shell: πακέτο msi. Δεν περιλαμβάνεται στην εγκατάσταση του διακομιστή.

Έλεγχος

Σε τερματικό: mongosh. Η προτροπή test> σημαίνει ότι όλα λειτουργούν.

macOS ή Linux

[MongoDB Manual: Install MongoDB Community Edition](#), οδηγίες ανά σύστημα.

Το παλαιό κέλυφος mongo δεν υπάρχει πλέον, από την έκδοση 6.0. Οδηγοί που το χρησιμοποιούν είναι παλιοί.

Εγκατάσταση τώρα

Στον υπολογιστή σας, με τα αρχεία που κατεβάσατε πριν από το διάλειμμα:

- 1 Εκτελέστε το `msi` του διακομιστή: **Complete, «Install MongoDB as a Service»**.
- 2 Εκτελέστε το `msi` του `mongosh`.
- 3 Ανοίξτε τερματικό και γράψτε `mongosh`. Η προτροπή `test>` σημαίνει επιτυχία.
- 4 Γράψτε στο chat του Zoom «OK», ή το μήνυμα λάθους που βλέπετε.

Όποιος δεν τελειώσει, συνεχίζει παράλληλα με το μάθημα· οι διαφάνειες δείχνουν κάθε έξοδο.

08:00

Τι είναι η MongoDB

2009

η πρώτη έκδοση, από την εταιρεία 10gen, σήμερα MongoDB Inc.

Βάση εγγράφων: αποθηκεύει έγγραφα τύπου JSON σε συλλογές.

- Η πιο διαδεδομένη βάση NoSQL: πέμπτη στη συνολική κατάταξη DB-Engines.
- Δωρεάν έκδοση Community για τον δικό σας υπολογιστή· υπηρεσία στο νέφος (Atlas) με δωρεάν επίπεδο.
- Νέα κύρια έκδοση περίπου κάθε χρόνο. Οι εντολές του μαθήματος ισχύουν σε όλες τις σύγχρονες εκδόσεις.

Βάση, συλλογή, έγγραφο

Βάση (database)

Περιέκτης συλλογών. Ένας διακομιστής φιλοξενεί πολλές βάσεις.



Συλλογή (collection)

Ομάδα εγγράφων με παρόμοιο σκοπό. Αντιστοιχεί στον πίνακα.



Έγγραφο (document)

Ζεύγη πεδίου-τιμής. Αντιστοιχεί στη γραμμή.

Χώρος ονομάτων (namespace): βάση.συλλογή

```
world.country  
library.books
```

Απόψε: βάση world, συλλογή country. Στο επόμενο μάθημα: library.books, από το αρχείο books.json.

Αντιστοιχία όρων με τις σχεσιακές βάσεις

Σχεσιακή βάση	MongoDB
Βάση δεδομένων	Βάση δεδομένων (database)
Πίνακας	Συλλογή (collection)
Γραμμή	Έγγραφο (document)
Στήλη	Πεδίο (field)
Πρωτεύον κλειδί	Το πεδίο <code>_id</code> , που δημιουργείται αυτόματα αν δεν το δώσουμε
Σύνδεση πινάκων (JOIN)	Εμφωλευμένο έγγραφο ή σύνδεση συλλογών με <code>\$lookup</code>

Η αντιστοιχία βοηθά στην αρχή. Ο σχεδιασμός όμως διαφέρει: ένα έγγραφο μπορεί να περιέχει ό,τι στη σχεσιακή βάση θα ήταν τρεις πίνακες.

JSON σε ένα λεπτό

```
{  
  "first_name": "John",  
  "last_name": "Doe",  
  "age": 22,  
  "skills": ["Programming",  
             "Databases", "API"]  
}
```

- `{ }`: έγγραφο (αντικείμενο)· `[ ]`: λίστα.
- Ζεύγη πεδίου–τιμής, χωρισμένα με κόμμα· τα ονόματα των πεδίων σε εισαγωγικά.
- Τιμές: κείμενο, αριθμοί, true και false, null, λίστες, έγγραφα.

BSON και τύποι δεδομένων

Τύπος	Στο mongosh	Σημείωση
String	"Greece"	κείμενο UTF-8
Int32	42	ακέραιος που χωρά σε 32 bit
Double	8.5 , 3000000000	δεκαδικός· και οι μεγάλοι ακέραιοι
Int64	NumberLong ("30000000000")	μεγάλος ακέραιος, ρητά
Decimal128	NumberDecimal ("0.10")	ακριβείς δεκαδικοί, π.χ. ποσά
Date	new Date ("2025-11-22")	ημερομηνία και ώρα, σε UTC

Η MongoDB αποθηκεύει κάθε έγγραφο σε BSON (Binary JSON): δυαδική μορφή, με τύπους που το JSON δεν έχει.

Το πεδίο `_id` και το `ObjectId`

12

bytes σε κάθε `ObjectId`: 4 για τη στιγμή δημιουργίας, 5 τυχαία, 3 μετρητής

Κάθε έγγραφο έχει μοναδικό `_id`. Αν δεν το δώσουμε, η MongoDB δημιουργεί ένα `ObjectId`.

- Το `_id` είναι το πρωτεύον κλειδί της συλλογής: μοναδικό και αμετάβλητο.
- Η στιγμή δημιουργίας διαβάζεται από το ίδιο το `ObjectId`: `.getTimestamp()`.
- Μπορούμε να δώσουμε δικό μας `_id`, αρκεί να είναι μοναδικό στη συλλογή.

Ενσωμάτωση ή αναφορά;

● Ενσωμάτωση (embedding)

- Τα δεδομένα διαβάζονται μαζί με το έγγραφο.
- Σχέση ένα-προς-λίγα, π.χ. οι διευθύνσεις ενός φοιτητή.
- Τα δεδομένα δεν αλλάζουν ανεξάρτητα.

● Αναφορά (reference)

- Τα δεδομένα αλλάζουν ή διαβάζονται και μόνα τους.
- Σχέση πολλά-προς-πολλά, ή λίστα χωρίς όριο: ένα έγγραφο δεν ξεπερνά τα 16 MB.
- Αποθηκεύουμε το `_id` του άλλου εγγράφου.

Ο κανόνας του εγχειριδίου της MongoDB: ό,τι χρησιμοποιείται μαζί, αποθηκεύεται μαζί.

Από τους πίνακες της Εργασίας 1 στα έγγραφα

```
// συλλογή students
{ _id: 1042,
  name: "Μαρία Παπαδοπούλου",
  enrolments: [
    { course_id: "DB101", grade: 8.5 },
    { course_id: "LA201", grade: 9 } ] }

// συλλογή courses
{ _id: "DB101",
  title: "Βάσεις Δεδομένων" }
```

- Οι εγγραφές ενσωματώνονται στον φοιτητή: διαβάζονται πάντα μαζί του.
- Τα μαθήματα μένουν σε δική τους συλλογή, με αναφορά μέσω του `course_id`: αλλάζουν ανεξάρτητα.
- Αυτό ζητά η Εργασία 2: δύο τουλάχιστον πίνακες της Εργασίας 1, ο ένας εμφωλευμένος στον άλλον.

Πλοήγηση: `db`, `show dbs`, `use`, `show collections`

```
test> db
test
test> show dbs
admin    40.00 KiB
config  12.00 KiB
local    40.00 KiB
test> use world
switched to db world
world> show collections
world>
```

- `db`: η τρέχουσα βάση. Το mongosh ξεκινά στη βάση test.
- `show dbs`: οι βάσεις του διακομιστή. Η world δεν υπάρχει ακόμη.
- `use world`: αλλαγή στη βάση world, που δημιουργείται με την πρώτη εισαγωγή.
`show collections`: καμία συλλογή.

Εισαγωγή ενός εγγράφου: insertOne

Εντολή

```
db.country.insertOne({  
  name: "Greece",  
  euSince: 1981,  
  population: 10400000  
})
```

Αποτέλεσμα

```
{  
  acknowledged: true,  
  insertedId: ObjectId('6ab3d2b0ff0f72f28211357d')  
}
```

- Χωρίς `_id`: η MongoDB δημιουργεί ένα `ObjectId`.
- `acknowledged: true`: ο διακομιστής επιβεβαίωσε την εγγραφή.
- Τώρα το `show dbs` δείχνει και τη `world`.

Δικό μας _id

Εντολή

```
db.country.insertOne({
  _id: 2,
  name: "Cyprus",
  population: 980000
})
```

Αποτέλεσμα

```
{ acknowledged: true, insertedId: 2 }
```

- Το `_id` το δώσαμε εμείς: η MongoDB δεν δημιουργεί `ObjectId`.
- Η Κύπρος δεν έχει πεδίο `euSince`: ίδια συλλογή, άλλο σχήμα.
- Δεύτερο έγγραφο με `_id: 2` απορρίπτεται, με σφάλμα διπλού κλειδιού.

Εισαγωγή πολλών: insertMany

Εντολή

```
db.country.insertMany([
  { name: "France", euSince: 1958, population: 68600000,
    capital: { name: "Paris", timezone: "Europe/Paris" } },
  { name: "Germany", euSince: 1958, population: 83600000,
    capital: { name: "Berlin", timezone: "Europe/Berlin" } }
])
```

Αποτέλεσμα

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab3d2b0ff0f72f28211357e'),
    '1': ObjectId('6ab3d2b0ff0f72f28211357f')
  }
}
```

- Μια λίστα εγγράφων σε μία εντολή.
- Τα δύο νέα έγγραφα έχουν πεδίο capital, που οι δύο πρώτες χώρες δεν έχουν.
- Ένα _id για κάθε έγγραφο, με τη σειρά εισαγωγής.
- Παλαιοί οδηγοί γράφουν `insert()`: λειτουργεί, με προειδοποίηση απόσυρσης.

Εμφωλευμένο έγγραφο

Η τιμή ενός πεδίου μπορεί να είναι άλλο έγγραφο: εμφωλευμένο (embedded) έγγραφο.

MongoDB Manual: Embedded Documents

Στο παράδειγμα

Η πρωτεύουσα είναι μέρος του εγγράφου της Γαλλίας: `capital: { name: "Paris", ... }`.

Για την Εργασία 2

Τουλάχιστον 4 έγγραφα, τα 2 με εμφωλευμένα έγγραφα, από τα δεδομένα της Εργασίας 1.

Η τελεία στο όνομα πεδίου

Εντολή

```
db.country.insertOne({ name: "Italy", "capital.name": "Rome" })
db.country.findOne({ name: "Italy" })
```

Αποτέλεσμα του findOne

```
{
  _id: ObjectId('6ab3d2b0ff0f72f282113580'),
  name: 'Italy',
  'capital.name': 'Rome'
}
```

- Με τελεία στο όνομα, η εισαγωγή δημιουργεί ένα πεδίο με όνομα «capital.name», όχι εμφωλευμένο έγγραφο.
- Το `find({ "capital.name": "Rome" })` δεν επιστρέφει τίποτε: στα ερωτήματα η τελεία σημαίνει διαδρομή.
- Σωστά: `capital: { name: "Rome" }`.

Ανάγνωση: findOne και find

Εντολή

```
db.country.findOne()
```

Αποτέλεσμα

```
{
  _id: ObjectId('6ab3d2b0ff0f72f28211357d'),
  name: 'Greece',
  euSince: 1981,
  population: 10400000
}
```

- `findOne()` : ένα έγγραφο, το πρώτο της συλλογής.
- `find()` : όλα τα έγγραφα. Το `mongosh` τα δείχνει ανά 20· με `it` εμφανίζονται τα επόμενα.
- Και οι δύο δέχονται φίλτρο.

Φίλτρο ισότητας

Εντολή

```
db.country.find({ name: "Greece" })
```

Αποτέλεσμα

```
[
  {
    _id: ObjectId('6ab3d2b0ff0f72f28211357d'),
    name: 'Greece',
    euSince: 1981,
    population: 10400000
  }
]
```

- Το φίλτρο είναι κι αυτό έγγραφο: `{ πεδίο: τιμή }`.
- Το find επιστρέφει λίστα, ακόμη κι όταν ταιριάζει ένα μόνο έγγραφο.
- Στην SQL: `WHERE name = 'Greece'`.

Σημειογραφία τελείας

Εντολή

```
db.country.find({ "capital.name": "Paris" })
```

Αποτέλεσμα

```
[
  {
    _id: ObjectId('6ab3d2b0ff0f72f28211357e'),
    name: 'France',
    euSince: 1958,
    population: 68600000,
    capital: { name: 'Paris', timezone: 'Europe/Paris' }
  }
]
```

- Με την τελεία το ερώτημα μπαίνει μέσα στο εμφωλευμένο έγγραφο.
- Το όνομα με την τελεία γράφεται πάντα σε εισαγωγικά.
- Ίδια σύνταξη και για λίστες εγγράφων:
"enrolments.course_id".

Πλήθος: `countDocuments`

Εντολές

```
db.country.countDocuments()  
db.country.countDocuments({ euSince: 1958 })
```

Αποτελέσματα

```
4  
2
```

- Χωρίς φίλτρο: όλα τα έγγραφα.
- Με φίλτρο: όσα ταιριάζουν, εδώ οι δύο ιδρυτικές χώρες.
- Το παλαιό `count ( )` λειτουργεί με προειδοποίηση απόσυρσης.

Διαγραφή: deleteOne και deleteMany

Εντολή

```
db.country.deleteOne({ name: "Cyprus" })
```

Αποτέλεσμα

```
{ acknowledged: true, deletedCount: 1 }
```

- `deleteOne`: το πρώτο έγγραφο που ταιριάζει στο φίλτρο.
- `deleteMany`: όλα όσα ταιριάζουν.
- **Προσοχή**: το `deleteMany({})` αδειάζει ολόκληρη τη συλλογή, χωρίς επιβεβαίωση.

Τι επιστρέφει αυτό το ερώτημα;

Η Γαλλία έχει πεδίο `capital` με εμφωλευμένο έγγραφο: `name` «Paris» και `timezone` «Europe/Paris». Τι επιστρέφει το `find({ capital: "Paris" });`

- A** Το έγγραφο της Γαλλίας
- B** Τίποτε
- Γ** Σφάλμα σύνταξης
- Δ** Όλα τα έγγραφα με πεδίο `capital`

01:00

Η απάντηση: τίποτε

Εντολές

```
db.country.find({ capital: "Paris" })  
db.country.find({ "capital.name": "Paris" })
```

Αποτέλεσμα του δεύτερου ερωτήματος

```
[  
  {  
    _id: ObjectId('6ab3d2b0ff0f72f28211357e'),  
    name: 'France',  
    euSince: 1958,  
    population: 68600000,  
    capital: { name: 'Paris', timezone: 'Europe/Paris' }  
  }  
]
```

- Το πρώτο ζητά capital ίσο με το κείμενο "Paris". Η τιμή όμως είναι έγγραφο: καμία αντιστοιχία, κανένα σφάλμα.
- Το δεύτερο μπαίνει στο εμφωλευμένο έγγραφο με την τελεία.

Εργασία 2: τι καλύψαμε απόψε

● Απόψε

- 1. Δημιουργία βάσης
- 2. Δημιουργία συλλογής
- 3. Δημιουργία εγγράφων: τουλάχιστον 4, τα 2 με εμφωλευμένα
- 12. Διαγραφή εγγράφου

● Στο επόμενο μάθημα

- 4. Προβολή πεδίων · 5. Λογικοί τελεστές · 6. Τελεστές σύγκρισης
- 7. Αναζήτηση με regex · 8. Σύνθετα ερωτήματα
- 9. Ταξινόμηση και περιορισμός · 10. Συνάθροιση (pipeline) · 11. Ενημέρωση

Η εκφώνηση ζητά επίσης, σε κάποια ερωτήματα, προβολή σε ευανάγνωστο JSON και χρήση μεταβλητών: στο επόμενο μάθημα.

Τρία σημεία από το Μέρος 3

- 1 Βάση, συλλογή, έγγραφο: η MongoDB αποθηκεύει έγγραφα BSON, καθένα με μοναδικό `_id`.
- 2 Ό,τι διαβάζεται μαζί ενσωματώνεται· ό,τι αλλάζει ανεξάρτητα ή μεγαλώνει χωρίς όριο, μπαίνει με αναφορά.
- 3 Στα ερωτήματα, η τελεία οδηγεί μέσα στο εμφωλευμένο έγγραφο. Στην εισαγωγή, γράφουμε έγγραφο μέσα σε έγγραφο.

Την επόμενη φορά

Ερωτήματα με τελεστές, ταξινόμηση, ενημέρωση και συνάθροιση, στο αρχείο books.json. Και ομαδική άσκηση σε αίθουσες του Zoom.